

Rapport de stage Spécialité : Informatique

Développement d'un middleware léger pour l'orchestration d'APIs
CBS avec supervision et documentation intégrée

Réalisé par : Abker Ranim

Entreprise : IntechGeeks



Encadrant : Wael Saidi

Période de stage : 01/07/2025 - 31/08/2025

Table des matières

Liste des abréviations	vi
Introduction générale	1
1 Contexte général	3
1.1 Organisme d'accueil	4
1.1.1 Présentation d'IntechGeeks	4
1.1.2 Valeurs fondamentales	4
1.1.3 Structure organisationnelle	5
1.2 Présentation du sujet	5
1.2.1 Contexte général du projet	5
1.3 Problématique	6
1.4 Étude de l'existant	6
1.4.1 Limites de la solution	7
1.5 Solution proposée	7
1.6 Méthodologie adoptée	8
1.6.1 Choix de la méthodologie de travail	8
1.6.2 Répartition des rôles	8
2 Analyse et Spécification des besoins	11
2.1 Spécification des acteurs	12
2.1.1 Identification des acteurs	12
2.2 Spécification des besoins	13
2.2.1 Besoins fonctionnels	13
2.2.2 Besoins non fonctionnels	16

2.2.3	Backlog de Produit	18
2.2.4	Planification des sprints	23
2.3	Analyse globale du projet	24
2.3.1	Diagramme de cas d'utilisation général	24
2.3.2	Diagramme de classes d'analyse	32
2.4	Architecture du système	34
2.4.1	Architecture physique	34
2.4.2	Architecture logique	35
2.5	Environnement de développement	35
2.5.1	Environnement logiciel	35
2.5.2	Technologies utilisées	37
Références bibliographiques		39

Table des figures

1.1	Logo d’IntechGeeks	4
1.2	Cycle de vie SCRUM	10
2.1	Diagramme de cas d’utilisation — SuperAdmin	25
2.2	Diagramme de cas d’utilisation — TechUser	26
2.3	Diagramme de cas d’utilisation — FrontOffice	27
2.4	Diagramme de cas d’utilisation — CRO	28
2.5	Diagramme de cas d’utilisation — BranchDM	29
2.6	Diagramme de cas d’utilisation — HeadOfficeDM	30
2.7	Diagramme de cas d’utilisation — RiskAnalyst	31
2.8	Diagramme de cas d’utilisation — ReadOnly	32
2.9	Diagramme de classes global — CrediWise	33

Liste des tableaux

- 1.1 Backlog de Produit 22
- 1.2 Planification des sprints 23
- 1.3 Environnement logiciel 37

Liste des abréviations

ACM	Autorité de Contrôle de la Microfinance
API	Application Programming Interface
CBS	Core Banking System
CIN	Carte d'Identité Nationale
CRA	Credit Risk Analysis
CRM	Customer Relationship Management
ERP	Enterprise Resource Planning
gRPC	Google Remote Procedure Call
IA	Intelligence Artificielle
JWT	JSON Web Token
OCR	Optical Character Recognition
PDF	Portable Document Format
REST	Representational State Transfer
SFO	Suivi Financier et Opérationnel
SPA	Single Page Application

Introduction générale

La gestion du crédit est un processus critique pour les établissements financiers, englobant l'entrée en relation client, la préparation des dossiers, l'évaluation des risques, la prise de décision et la traçabilité. Dans de nombreux environnements opérationnels, ces activités sont réparties entre des outils hétérogènes ou reposent sur des plateformes low-code qui imposent des contraintes en matière de scalabilité, d'interopérabilité et de maintenabilité à long terme.

Une analyse de DigiLAF, la solution de gestion du crédit existante déployée chez l'entreprise d'accueil, a révélé que si elle numérise avec succès le workflow central du crédit, son architecture introduit des limitations en matière de scalabilité, de flexibilité vis-à-vis des fournisseurs et d'auditabilité qui deviennent critiques à grande échelle. Ces contraintes motivent la conception d'une plateforme sur mesure construite sur des technologies ouvertes et maintenables.

Pour répondre à ces besoins, ce projet propose **CrediWise**, une plateforme web sécurisée organisée autour de domaines métier indépendants. Le rapport présente l'analyse, la conception et l'implémentation de la plateforme en dix chapitres, chaque chapitre-sprint correspondant soit à la fondation DevOps, soit à un ou plusieurs microservices de l'architecture cible alignés sur une phase du workflow de demande de crédit :

- **Chapitre 1 : Contexte du Projet** — présente l'entreprise d'accueil, le contexte du projet, la solution existante, la solution proposée et la méthodologie.
- **Chapitre 2 : Analyse et spécification des besoins** — Ce chapitre aborde la définition des exigences fonctionnelles et non fonctionnelles. Nous y présentons en détail la planification des sprints ainsi que la gestion du backlog produit, tout en décrivant l'architecture technique de l'application.
- **Chapitre 3 : Sprint 1, Authentification et gestion de la clientèle** — présente la base de sécurité fondée sur Keycloak, ainsi que l'administration des agences, des gestionnaires et des rôles. Il introduit également la gestion des dossiers clients, les

données de référence et l'exposition du domaine client aux autres services via les mécanismes d'interconnexion entre services.

- **Chapitre 4 : Sprint 2, Service de Demande de Crédit** — détaille la création des demandes, la gestion des brouillons, le routage basé sur les produits et les transitions d'état du cycle de vie.
- **Chapitre 5 : Sprint 3, Service d'Analyse** — décrit le dossier d'analyse, le workflow par étapes et le calcul du scoring pour les produits éligibles à l'analyse.
- **Chapitre 6 : Sprint 4, Service Checklist** — présente la checklist pré-comité qui consolide les documents et les sections de vérification requis avant la phase comité.
- **Chapitre 7 : Sprint 5, Service d'Analyse des Risques** — décrit le formulaire d'analyse du risque de crédit (CRA), les modules d'avis et de Q&R, ainsi que la piste d'audit associée.
- **Chapitre 8 : Sprint 6, Services du comité de crédit et de décaissement** — présente le procès-verbal de décision du comité ainsi que le procès-verbal de décaissement, et la clôture du cycle de vie de la demande de crédit.
- **Chapitre 9 : Sprint 0, Fondation DevOps et Observabilité** — établit le pipeline CI/CD Azure DevOps, l'automatisation de build Maven, la génération d'images Docker, la topologie docker-compose de production avec le reverse proxy Nginx, l'instrumentation OpenTelemetry de chaque service backend, et la stack d'observabilité SigNoz utilisée pour le tracing distribué, les métriques et les logs centralisés.
- **Chapitre 10 :** —

Contexte général

Plan

1	Organisme d'accueil	4
2	Présentation du sujet	5
3	Problématique	6
4	Étude de l'existant	6
5	Solution proposée	7
6	Méthodologie adoptée	8

Introduction

Ce chapitre présente le contexte général du projet CrediWise. Il introduit l'entreprise d'accueil IntechGeeks, décrit le secteur de la microfinance en Tunisie, qui motive le projet, analyse les points forts et les limites de la solution existante DigiLAF, présente la plateforme CrediWise proposée et expose la méthodologie Scrum adoptée pour le développement.

1.1 Organisme d'accueil

1.1.1 Présentation d'IntechGeeks

IntechGeeks est une société d'ingénierie et d'intelligence artificielle au service de clients en Europe et en Afrique. Ses activités couvrent l'architecture logicielle cloud-native, l'intégration de systèmes et l'intelligence artificielle appliquée, avec un accent particulier sur la modernisation des systèmes d'information organisationnels. La figure 1.1 présente le logo de l'entreprise.



Figure 1.1 : Logo d'IntechGeeks

1.1.2 Valeurs fondamentales

Le travail de l'entreprise s'articule autour de principes qui guident la collaboration et la réalisation technique :

- **Rigueur technique** : Les systèmes livrés sont testés, documentés et instrumentés. Les tests automatisés, les pipelines reproductibles et l'observabilité sont considérés comme des critères essentiels de qualité du projet.

- **Immersion avant exécution** : Avant le début du développement, l'entreprise analyse le contexte opérationnel du client, les systèmes existants, les contraintes et les flux de travail, afin que la solution proposée corresponde aux besoins réels de l'entreprise.
- **Transfert de connaissances** : La livraison du projet comprend une documentation, l'explication des choix techniques et un accompagnement destiné à aider les équipes clientes à exploiter la solution livrée.
- **Transparence** : L'avancement du projet, les décisions techniques et les risques identifiés sont communiqués aux parties prenantes tout au long de la mission.

1.1.3 Structure organisationnelle

1.2 Présentation du sujet

Ce projet de fin d'études porte sur la conception et le développement de **CrediWise**, une plateforme web de gestion du crédit. L'objectif est de digitaliser et sécuriser l'ensemble du cycle de vie du crédit, de l'entrée en relation client jusqu'à la décision finale et la traçabilité.

1.2.1 Contexte général du projet

Le secteur de la microfinance en Tunisie est supervisé par l'Autorité de Contrôle de la Microfinance (ACM). Les principaux acteurs — AKDI, Al Amal Microfinance, Zitouna Tamkeen, Advans Tunisie, Taysir Microfinance et Enda Tamweel, entre autres — jouent un rôle essentiel dans l'inclusion financière et le soutien des micro-entrepreneurs.

Malgré la croissance du secteur, la gestion du crédit reste un processus complexe, fortement documentaire et encore peu digitalisé en Tunisie et en Afrique en général.

Dans ce contexte, IntechGeeks a initié **CrediWise** comme module de gestion du crédit intégré à la plateforme KAYA, un middleware cloud-native permettant l'interconnexion de systèmes financiers hétérogènes (Core Banking, CRM et ERP).

1.3 Problématique

Les solutions existantes de gestion du crédit souffrent de plusieurs limites, notamment en matière de scalabilité, de flexibilité et de maintenabilité.

Les processus restent souvent fragmentés, fortement dépendants de plateformes low-code propriétaires, ce qui entraîne :

- une limitation des performances sous forte charge,
- un coût élevé d'exploitation,
- une faible interopérabilité avec les systèmes externes,
- une traçabilité insuffisante des décisions,
- et une absence d'automatisation avancée (scoring, règles métier).

Ces contraintes rendent nécessaire la conception d'une solution moderne, modulaire et évolutive capable de supporter des processus de crédit complexes.

1.4 Étude de l'existant

La solution étudiée dans ce projet est **DigiLAF**, une application de gestion des crédits développée avec Microsoft Power Apps (Canvas App) et déployée dans plusieurs institutions financières en Tunisie et en Afrique.

Elle digitalise l'ensemble du cycle de crédit, de la création de la demande jusqu'au déblocage final, via un workflow basé sur le Business Process Flow.

Le processus comprend cinq étapes principales :

- **Analyse** : évaluation initiale et génération d'un rapport préliminaire.
- **Checklist** : vérification des documents requis.
- **Analyse des risques** : évaluation de la solvabilité et du risque.
- **Revue comité** : validation et décision du comité de crédit.
- **Visite de gestion** : contrôle terrain si nécessaire.

Power Automate est utilisé pour automatiser les transitions et les notifications, tandis que la gestion des accès repose sur les rôles de la Power Platform.

1.4.1 Limites de la solution

Malgré ses avantages, DigiLAF présente plusieurs limitations :

- scalabilité limitée,
- coût élevé (licences et connecteurs),
- dépendance forte à Microsoft,
- limitation de volume de données (2 000 enregistrements par requête),
- fragmentation du workflow,
- absence d'IA et de scoring automatique,
- faible observabilité et traçabilité,
- absence de portail client.

1.5 Solution proposée

Pour répondre à ces limites, ce projet propose **CrediWise**, une plateforme web de gestion du crédit basée sur une architecture modulaire et indépendante des solutions low-code.

La plateforme centralise la gestion des clients, des demandes de crédit, des analyses, des décisions et de la traçabilité dans une interface unifiée.

Fonctionnalités principales

- **Administration des utilisateurs et agences** : gestion des rôles et profils.
- **Gestion des clients** : centralisation des données clients.
- **Gestion des demandes de crédit** : création, brouillons, garanties et export PDF.
- **Routage par produit** : orientation automatique du workflow.
- **Analyse du crédit** : dossier structuré avec étapes d'évaluation.
- **Scoring et règles métier** : aide à la décision et traçabilité.

- **Modules de contrôle** : checklist, risques et comité de crédit.
- **Suivi du décaissement** : clôture du cycle de vie du crédit.

Sur le plan architectural, CrediWise repose sur une séparation claire entre frontend, backend, gestion d'identité (Keycloak) et bases de données. Cette architecture améliore la scalabilité, la maintenabilité et la traçabilité, tout en répondant aux limites de la solution existante.

1.6 Méthodologie adoptée

La réussite d'un projet informatique repose sur une méthodologie de travail structurée. Celle-ci permet d'organiser les activités, de respecter les délais et de garantir la qualité des livrables tout au long du cycle de développement.

1.6.1 Choix de la méthodologie de travail

Afin d'assurer une gestion efficace du projet, nous avons adopté la méthodologie **Agile Scrum** [1]. Cette approche favorise une livraison progressive des fonctionnalités tout en permettant une adaptation continue aux besoins.

Les principaux avantages de Scrum sont :

- Une meilleure visibilité sur l'avancement du projet.
- Une implication régulière des parties prenantes.
- Une priorisation des besoins selon leur importance.
- Une organisation du travail en sprints, facilitant le suivi du projet.

1.6.2 Répartition des rôles

Une équipe Scrum est composée du Product Owner, du Scrum Master et de l'équipe de développement. Ces rôles complémentaires collaborent afin d'assurer le bon déroulement du projet et la livraison régulière de fonctionnalités à valeur ajoutée.

1.6.2.1 Scrum Master

Le Scrum Master veille à l'application des principes Scrum et accompagne l'équipe dans leur mise en œuvre. Il facilite la communication et contribue à la résolution des obstacles rencontrés durant le projet.

1.6.2.2 Product Owner

Le Product Owner est responsable de la gestion du Product Backlog et de la priorisation des besoins. Il assure la liaison entre les parties prenantes et l'équipe de développement afin de garantir l'adéquation de la solution aux besoins métier.

1.6.2.3 Équipe de Développement

L'équipe de développement regroupe les membres chargés de concevoir, développer et tester la solution. À chaque sprint, elle produit un incrément fonctionnel respectant les critères de qualité définis.

1.6.2.4 Les événements de Scrum

La méthodologie Scrum repose sur plusieurs événements permettant de suivre l'avancement du projet :

- **Sprint Planning** : définition des objectifs et des tâches du sprint.
- **Daily Scrum** : réunion quotidienne de suivi et de coordination.
- **Sprint Review** : présentation et validation des fonctionnalités réalisées.
- **Sprint Retrospective** : identification des améliorations à apporter aux prochains sprints.
- **Backlog Refinement** : clarification et estimation des éléments du Product Backlog.

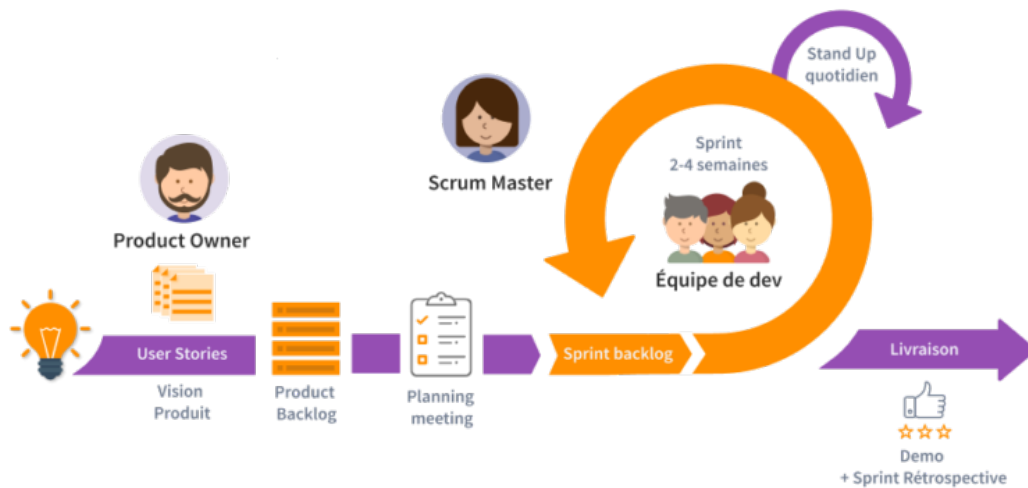


Figure 1.2 : Cycle de vie SCRUM

Conclusion

Ce chapitre débute par une présentation de l'organisme d'accueil. Par la suite, nous avons exposé une vue globale du projet, incluant la description de la solution actuelle, ses limites, la problématique identifiée, ainsi que la solution envisagée. Enfin, nous avons expliqué la méthodologie de gestion de projet adoptée. Le chapitre suivant se focalisera sur l'analyse et la spécification des besoins du projet.

Analyse et Spécification des besoins

Plan

1	Spécification des acteurs	12
2	Spécification des besoins	13
3	Analyse globale du projet	24
4	Architecture du système	34
5	Environnement de développement	35

Introduction

Dans ce chapitre, nous identifions les caractéristiques du projet en analysant les besoins fonctionnels et non fonctionnels. Nous définissons également les acteurs et clarifions les exigences, qui sont ensuite représentées à travers un diagramme global de cas d'utilisation. Enfin, nous présentons l'organisation du développement en sprints.

2.1 Spécification des acteurs

2.1.1 Identification des acteurs

L'identification des acteurs permet de définir les périmètres d'accès à l'application. Dans CrediWise, les acteurs correspondent à des profils opérationnels définis dans Keycloak et propagés au frontend et au backend via un mécanisme d'autorisation basé sur les rôles.

Les acteurs suivants ont été identifiés :

- **Super Administrateur** : dispose du niveau d'accès le plus élevé. Il gère les agences, les comptes gestionnaires, les rôles, les règles d'affichage ainsi que les données administratives du système.
- **Utilisateur technique** : assure des tâches d'administration technique, principalement liées à la gestion des agences et des comptes gestionnaires, sans intervenir dans les processus métier de décision.
- **Chargé de relation client (CRO)** : consulte les dossiers clients, suit les demandes de crédit et complète les dossiers d'analyse lorsqu'ils lui sont affectés.
- **Agent front office** : assure l'accueil des clients, crée les fiches clients, prépare les demandes de crédit et soumet les brouillons pour traitement.
- **Décideur agence** : examine les demandes de crédit et les dossiers d'analyse au niveau de l'agence. Il participe aux décisions et aux transitions de statut selon les règles d'autorisation.
- **Décideur siège** : consulte les dossiers nécessitant une validation centrale et participe

au processus de décision pour les demandes hors périmètre agence.

- **Analyste risque** : analyse les demandes de crédit, les données d'évaluation et les résultats de scoring afin de contribuer à l'analyse des risques.
- **Auditeur** : dispose d'un accès en lecture seule, utilisé pour les activités d'audit, de contrôle et de vérification.
- **Fournisseur d'identité (Keycloak)** : acteur système externe chargé de l'authentification, de la gestion des sessions et de l'émission des jetons. Il fournit également les rôles consommés par le frontend et le backend.

2.2 Spécification des besoins

Dans cette section, nous analysons les besoins fonctionnels et non fonctionnels du système, constituant une étape essentielle dans la définition de l'architecture du projet.

2.2.1 Besoins fonctionnels

➤ Authentification et sécurité

- Le système authentifie les utilisateurs via Keycloak et récupère les jetons d'accès à l'aide de l'adaptateur keycloak-js.
- Le frontend protège les routes en fonction des rôles de l'utilisateur, tandis que le backend applique les autorisations via l'annotation *@RolesAllowed*.
- Les jetons d'accès sont attachés aux requêtes API internes et renouvelés automatiquement avant expiration afin d'assurer la continuité des sessions.
- La déconnexion est gérée par Keycloak, garantissant la fermeture complète de la session au niveau du fournisseur d'identité.

➤ Gestion des utilisateurs et des agences

- Le service Manager permet de créer, modifier, désactiver et consulter les comptes gestionnaires.

- Le système prend en charge la gestion complète des agences (création, mise à jour, consultation et activation/désactivation).
- Les gestionnaires peuvent être associés à des agences, avec une cohérence assurée entre les rôles Keycloak et les données applicatives.
- Chaque utilisateur peut consulter et mettre à jour son profil selon les droits définis par l'API.

➤ Gestion des clients

- Le service Customer gère les fiches clients (personnes physiques et morales).
- Les données incluent l'identité, les coordonnées, l'agence, le type de compte, le segment, le secteur d'activité, le scoring et le cycle client.
- Le système permet la recherche de clients par identifiant ou coordonnées, ainsi que la consultation paginée des listes.
- Les données de référence (segments, secteurs, activités, types de compte, etc.) sont exposées pour garantir une saisie structurée.

➤ Gestion des demandes de crédit

- Le service Credit Request permet la création et la mise à jour des demandes de crédit au statut brouillon.
- Une demande inclut les informations client, le montant, la durée, le produit, les garanties, les garants, la capacité de remboursement et les indicateurs de risque.
- Une fois soumise, la demande est routée selon le produit : les produits 101, 102 et 103 sont dirigés vers l'analyse, les autres vers la checklist pré-comité.
- Les acteurs autorisés peuvent mettre à jour le statut dans le cadre du cycle de vie du crédit.
- Une version PDF de la demande peut être générée pour archivage et documentation.

➤ Analyse du crédit et scoring

- Le service Analysis crée un dossier d'analyse pour chaque demande éligible.

- Le processus suit plusieurs étapes : informations client, objet du crédit, risques client, commerciaux et financiers, et données complémentaires.
- Les opérations de sauvegarde et de validation sont limitées aux profils autorisés.
- Un module de scoring calcule et enregistre le score de crédit associé à la demande.
- Les règles d’affichage sont versionnées afin de garantir la traçabilité des décisions.

➤ Liste de contrôle pré-comité

- Le service Checklist consolide les documents et vérifications nécessaires avant le passage en comité.
- La checklist est générée ou mise à jour à partir de la demande et enrichie via le service Customer.
- Elle couvre les documents, garanties, archivage, préparation au décaissement et contrôles front office/back office.

➤ Analyse du risque crédit

- Le service Risk Analysis gère le formulaire d’analyse du risque crédit (CRA).
- Le formulaire suit plusieurs statuts contrôlés : brouillon, en révision, validé, envoyé au comité, ou archivé.
- Les analystes peuvent compléter les sections d’avis et de questions-réponses.
- Un module d’audit enregistre l’historique des modifications.

➤ Décision du comité

- Le service Credit Committee enregistre et expose les décisions du comité de crédit.
- Les décisions sont initialisées à partir des services amont et mises à jour après validation.
- Elles influencent directement le cycle de vie de la demande (approbation, rejet ou décaissement).

➤ Décaissement

- Le service Disbursement enregistre les informations de décaissement pour les demandes approuvées.
- L'enregistrement est basé sur l'instantané de la demande de crédit.
- Une fois validé, le statut passe à « décaissé », et le compteur client est mis à jour via gRPC.

➤ Auditabilité et traçabilité

- Chaque service enregistre les métadonnées de création et de mise à jour des entités.
- La sécurité contrôle les accès aux routes API et aux services backend.
- Les utilisateurs en lecture seule peuvent consulter les données sans pouvoir les modifier, garantissant les besoins d'audit et de conformité.

2.2.2 Besoins non fonctionnels

➤ Performance

Le backend est implémenté avec Quarkus, offrant un temps de démarrage rapide et une faible consommation mémoire. Les API REST intègrent des mécanismes de pagination et de filtrage afin de limiter la taille des réponses. Le frontend, développé avec Vite et React, assure une navigation fluide et réactive en réduisant les rechargements de page.

➤ Scalabilité

L'architecture repose sur une séparation des principaux domaines métier en services indépendants : Manager, Customer, Credit Request et Analysis. Cette approche permet à chaque service, ainsi qu'à sa base de données associée, d'évoluer et d'être dimensionné indépendamment en fonction de la charge.

➤ **Fiabilité et disponibilité**

L'infrastructure locale est déployée via Docker Compose, incluant PostgreSQL, Keycloak et pgAdmin. La séparation des services et des bases de données limite l'impact des défaillances à un domaine spécifique et garantit la reproductibilité des environnements de développement.

➤ **Maintenabilité**

Chaque service backend est structuré comme un projet Maven indépendant avec son propre code, sa configuration, ses tests et ses migrations Flyway. Cette organisation réduit le couplage entre services et facilite la maintenance ainsi que l'évolution du système.

➤ **Conformité**

Le système traite des données personnelles et financières, ce qui impose des exigences strictes de traçabilité et de contrôle d'accès. L'utilisation d'instantanés immuables des demandes et de métadonnées d'audit garantit la cohérence des données et la conformité aux exigences réglementaires.

➤ **Sécurité**

L'authentification est centralisée via Keycloak. Les services backend valident les jetons JWT et extraient les rôles configurés pour appliquer les règles d'autorisation. Les contrôles d'accès sont appliqués au niveau des API et renforcés au niveau métier lorsque nécessaire. Les jetons ne sont pas stockés dans le stockage local du navigateur afin de réduire les risques de sécurité.

➤ **Ergonomie et expérience utilisateur**

Le frontend est une application monopage (SPA) développée avec React et TypeScript. Il propose une navigation basée sur les rôles, une gestion multilingue via i18next, une validation des formulaires et un système cohérent de gestion des erreurs afin d'améliorer l'expérience utilisateur.

2.2.3 Backlog de Produit

Après avoir défini les besoins fonctionnels de notre solution, le backlog de produit est présenté dans le tableau 1.1 ci-après :

Sprint	Récit Utilisateur	Priorité	Estimation
Sprint 1 : Authentification et gestion de la clientèle			
	En tant que gestionnaire, je veux me connecter à la plateforme avec mes identifiants Keycloak, afin d'accéder à mon espace de travail sécurisé.	1	8h
	En tant que gestionnaire, je veux que mon rôle détermine automatiquement mes droits d'accès.	1	10h
	En tant que super administrateur, je veux créer et gérer les comptes gestionnaires.	1	12h
	En tant que super administrateur, je veux créer et gérer les agences.	1	8h
	En tant que gestionnaire, je veux consulter et modifier mon profil, afin de maintenir mes données à jour.	2	5h
	En tant que gestionnaire, je veux créer et gérer un client.	1	12h
	En tant que gestionnaire, je veux rechercher et filtrer les clients par nom, CIN ou segment, afin de retrouver rapidement un dossier existant.	2	6h
	En tant que super administrateur, je veux suspendre ou réactiver un compte gestionnaire, afin d'appliquer les politiques RH de l'établissement.	3	4h

Sprint	Récit Utilisateur	Priorité	Estimation
Sprint 2 : Service de Demande de Crédit			
	En tant que gestionnaire front office, je veux créer une demande de crédit en brouillon pour un client.	1	12h
	En tant que gestionnaire, je veux prendre un instantané des données client au moment de la soumission, afin de conserver une trace immuable des informations utilisées.	1	8h
	En tant que gestionnaire, je veux soumettre la demande de crédit, afin de déclencher automatiquement son acheminement vers le bon service selon le produit choisi.	1	8h
	En tant que gestionnaire, je veux modifier une demande en statut brouillon.	2	5h
	En tant que gestionnaire, je veux consulter la liste de toutes les demandes.	2	6h
	En tant que gestionnaire, je veux supprimer une demande en statut brouillon.	3	3h
	En tant que système, je veux router automatiquement les produits 101/102/103 vers le dossier d'analyse et les produits 104/105 vers la checklist pré-comité.	1	8h
Sprint 3 : Service d'Analyse			
	En tant qu'analyste de crédit, je veux accéder au dossier d'analyse en 7 étapes associé à une demande soumise.	1	10h

Sprint	Récit Utilisateur	Priorité	Estimation
	En tant qu'analyste, je veux renseigner l'étape 1 (visite client) avec la date de visite, le lieu et les observations terrain.	1	8h
	En tant qu'analyste, je veux renseigner l'étape 2 (objet du crédit) avec le type d'actif et la finalité du financement.	1	6h
	En tant qu'analyste, je veux évaluer le risque client à l'étape 3, afin d'identifier les signaux d'alerte.	1	8h
	En tant qu'analyste, je veux évaluer le risque commercial à l'étape 4, afin d'apprécier la viabilité économique du projet.	2	8h
	En tant qu'analyste, je veux analyser les états financiers à l'étape 5 via extraction OCR automatique.	1	16h
	En tant qu'analyste, je veux évaluer les garanties proposées à l'étape 6, afin de quantifier la couverture du risque.	2	8h
	En tant qu'analyste, je veux rédiger la proposition finale à l'étape 7 avec la recommandation et les conditions suggérées.	1	8h
	En tant qu'administrateur, je veux gérer les règles d'affichage des étapes d'analyse (actif/inactif, version).	3	6h
Sprint 4 : Service Checklist			
	En tant que gestionnaire, je veux générer automatiquement la checklist associée à une demande de crédit.	1	8h

Sprint	Récit Utilisateur	Priorité	Estimation
	En tant que gestionnaire, je veux consulter et cocher les éléments de la checklist pré-comité section par section.	1	10h
	En tant que gestionnaire, je veux valider les éléments de la checklist de décaissement (documents signés, conditions levées).	1	8h
	En tant que gestionnaire, je veux valider les sections de la checklist SFO et contrôle, afin de respecter les procédures internes de vérification.	2	6h
	En tant que gestionnaire, je veux régénérer la checklist d'une demande, afin de réinitialiser les éléments en cas de modification du dossier.	3	4h
Sprint 5 : Service d'Analyse des Risques			
	En tant qu'analyste risque, je veux créer un formulaire CRA pour une demande, afin de formaliser l'évaluation du risque.	1	10h
	En tant qu'analyste risque, je veux renseigner les éléments de checklist du formulaire CRA par section.	1	8h
	En tant qu'analyste risque, je veux répondre aux questions d'assurance qualité (Q&A) du formulaire CRA.	1	8h
	En tant qu'analyste risque, je veux saisir des opinions de risque par thème (avec couleur et note).	1	10h

Sprint	Récit Utilisateur	Priorité	Estimation
	En tant qu'analyste risque, je veux consulter la piste d'audit du formulaire CRA (qui a fait quoi et quand).	2	6h
Sprint 6 : Services du comité de crédit et de décaissement			
	En tant que décideur, je veux accéder au dossier du comité de crédit pour une demande, afin de préparer la session de décision.	1	8h
	En tant que décideur, je veux enregistrer la décision du comité (APPROUVÉ, REJETÉ) avec le montant et la durée approuvés.	1	10h
	En tant que décideur, je veux saisir les conditions d'approbation et les observations du comité.	1	8h
	En tant que gestionnaire du comité, je veux renseigner le formulaire de préparation (nom du rapporteur, nom du décideur).	2	5h
	En tant que gestionnaire front office, je veux créer le dossier de décaissement pour une demande approuvée.	1	8h
	En tant que gestionnaire, je veux enregistrer les informations de décaissement (montant, date, référence Amplitude).	1	8h
	En tant que gestionnaire, je veux confirmer le décaissement effectif, afin de clôturer le cycle d'octroi.	1	6h

Tableau 1.1 : Backlog de Produit

2.2.4 Planification des sprints

Le système sera développé de manière itérative selon la méthodologie Scrum. Chaque sprint permettra de livrer progressivement de nouvelles fonctionnalités, contribuant à la construction du produit final. Le tableau 1.2 présente la planification des sprints :

Sprint	Libellé	Début prévu	Fin prévu
1	Conception générale et mise en place de l'infrastructure	16-02-2025	21-02-2025
	Authentification Keycloak et gestion de la clientèle	24-02-2025	07-03-2025
2	Modélisation et création des demandes de crédit	10-03-2025	19-03-2025
	Soumission, scoring et routage des demandes	20-03-2025	28-03-2025
3	Dossier d'analyse — étapes 1 à 4 (visite client, objet, risques)	31-03-2025	11-04-2025
	Analyse financière OCR et proposition finale — étapes 5 à 7	14-04-2025	25-04-2025
4	Checklist pré-comité et checklist de décaissement	28-04-2025	02-05-2025
	Checklist garanties, SFO et contrôles	05-05-2025	09-05-2025
5	Formulaire CRA — checklist risque et questions Q&A	12-05-2025	16-05-2025
	Opinions de risque, workflow CRA et piste d'audit	19-05-2025	23-05-2025
6	Comité de crédit — décision, dérogation et préparation	26-05-2025	06-06-2025
	Décaissement, clôture du dossier et recommandation IA	09-06-2025	16-06-2025

Tableau 1.2 : Planification des sprints

2.3 Analyse globale du projet

Une fois la planification du projet établie, nous procédons à la modélisation du projet à travers des diagrammes qui illustrent les différentes fonctionnalités de l'application.

2.3.1 Diagramme de cas d'utilisation général

Le diagramme de cas d'utilisation global représente les interactions entre les différents acteurs de la plateforme CrediWise et l'ensemble des fonctionnalités qu'elle expose.

➤ Diagramme de cas d'utilisation SuperAdmin

Le diagramme de cas d'utilisation SuperAdmin (Figure 2.1) présente les fonctionnalités accessibles à l'administrateur système. Il couvre la gestion des agences, des gestionnaires, des clients, des demandes de crédit, des dossiers d'analyse, de l'analyse de risque, du comité de crédit, du décaissement et des recommandations IA.

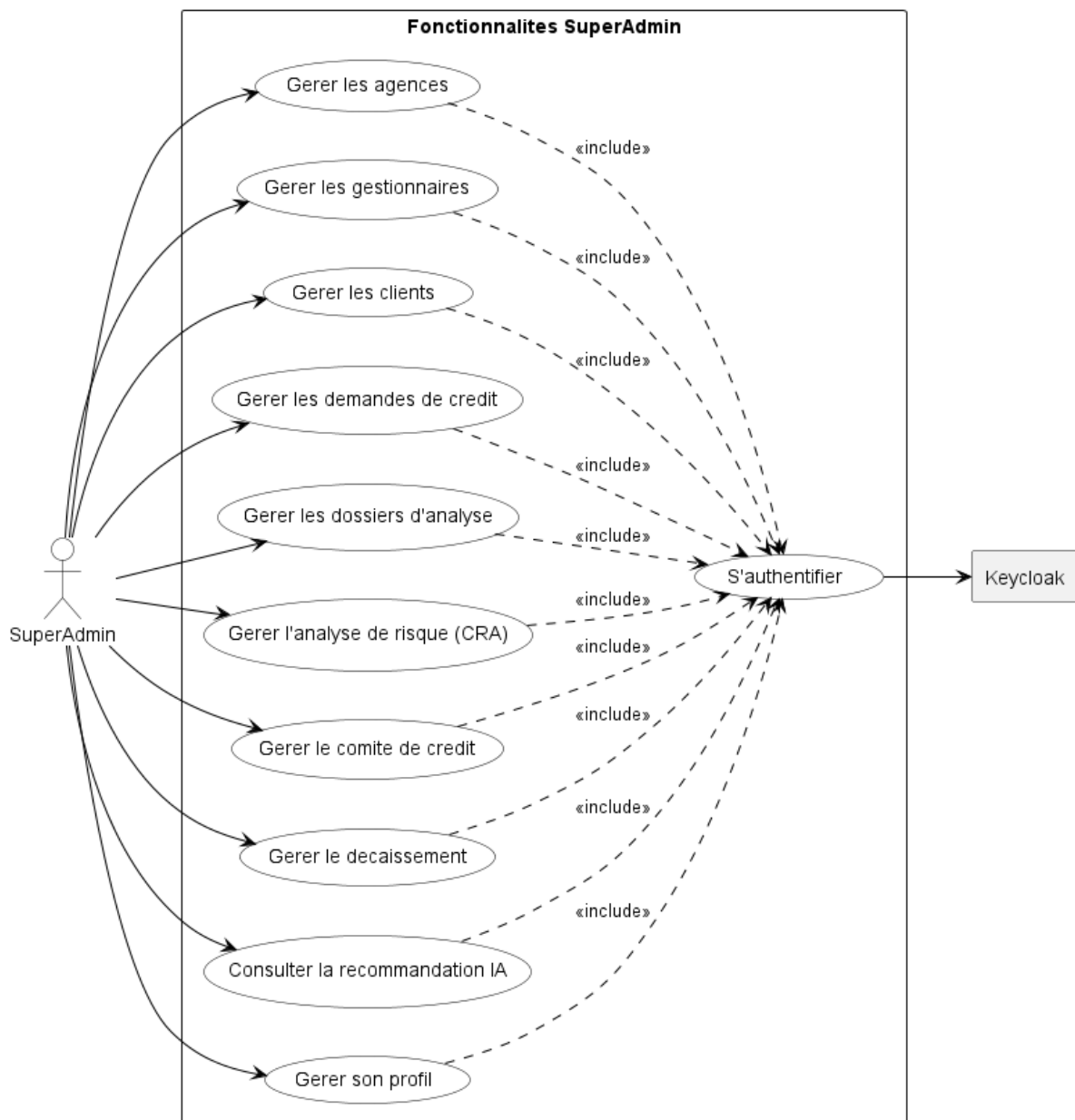


Figure 2.1 : Diagramme de cas d'utilisation — SuperAdmin

➤ Diagramme de cas d'utilisation TechUser

Le diagramme de cas d'utilisation TechUser (Figure 2.2) présente les fonctionnalités accessibles à l'utilisateur technique. Il inclut la gestion des gestionnaires ainsi que la consultation des agences, des clients, des demandes de crédit, des dossiers d'analyse, du comité de crédit et du décaissement.

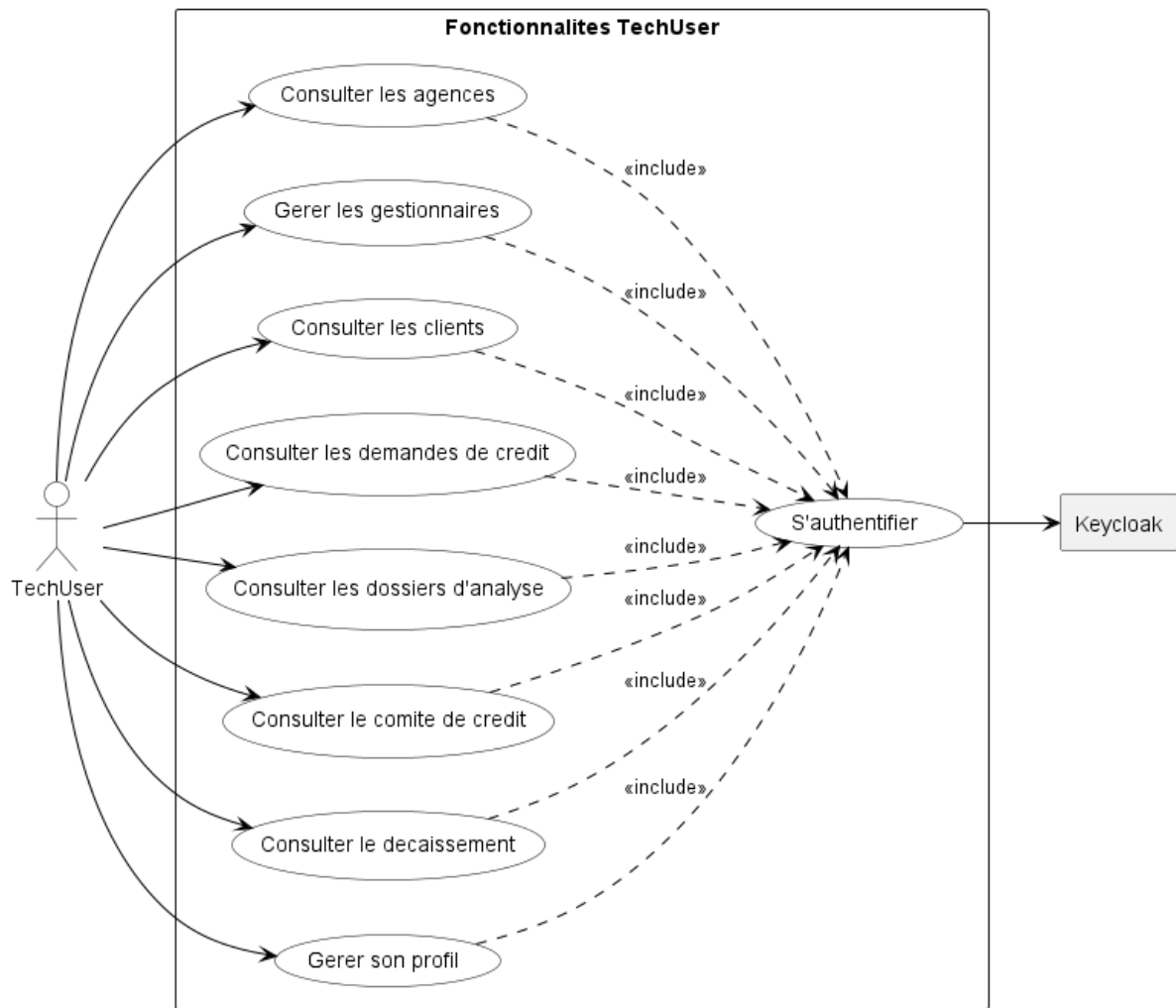


Figure 2.2 : Diagramme de cas d'utilisation — TechUser

➤ Diagramme de cas d'utilisation FrontOffice

Le diagramme de cas d'utilisation FrontOffice (Figure 2.3) présente les fonctionnalités accessibles au chargé de clientèle. Il inclut la gestion des clients, la création des demandes de crédit, la gestion des dossiers d'analyse, le déverrouillage du CRA, ainsi que le suivi du comité de crédit et du décaissement.

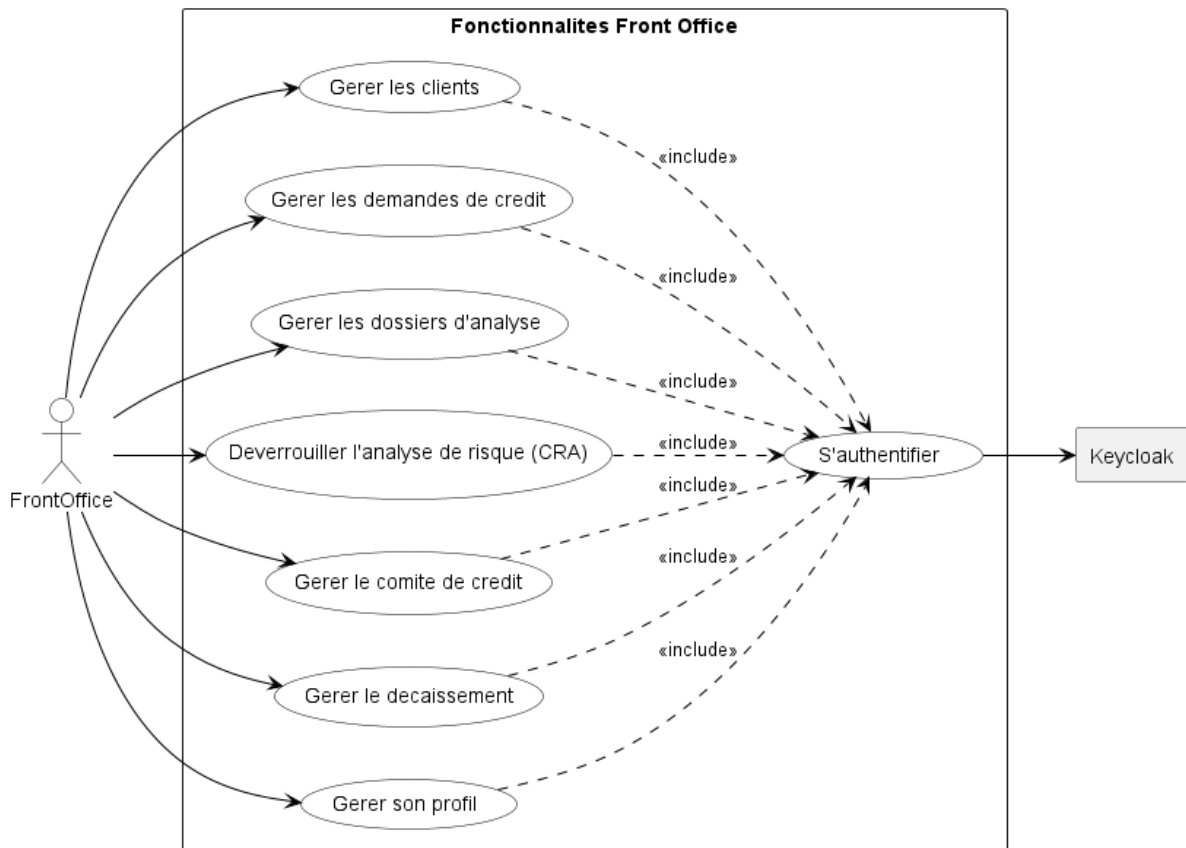


Figure 2.3 : Diagramme de cas d'utilisation — FrontOffice

➤ Diagramme de cas d'utilisation CRO

Le diagramme de cas d'utilisation CRO (Figure 2.4) présente les fonctionnalités accessibles au chargé des risques de crédit. Il inclut la gestion des demandes de crédit, des dossiers d'analyse, de l'analyse de risque CRA, du comité de crédit et du décaissement.

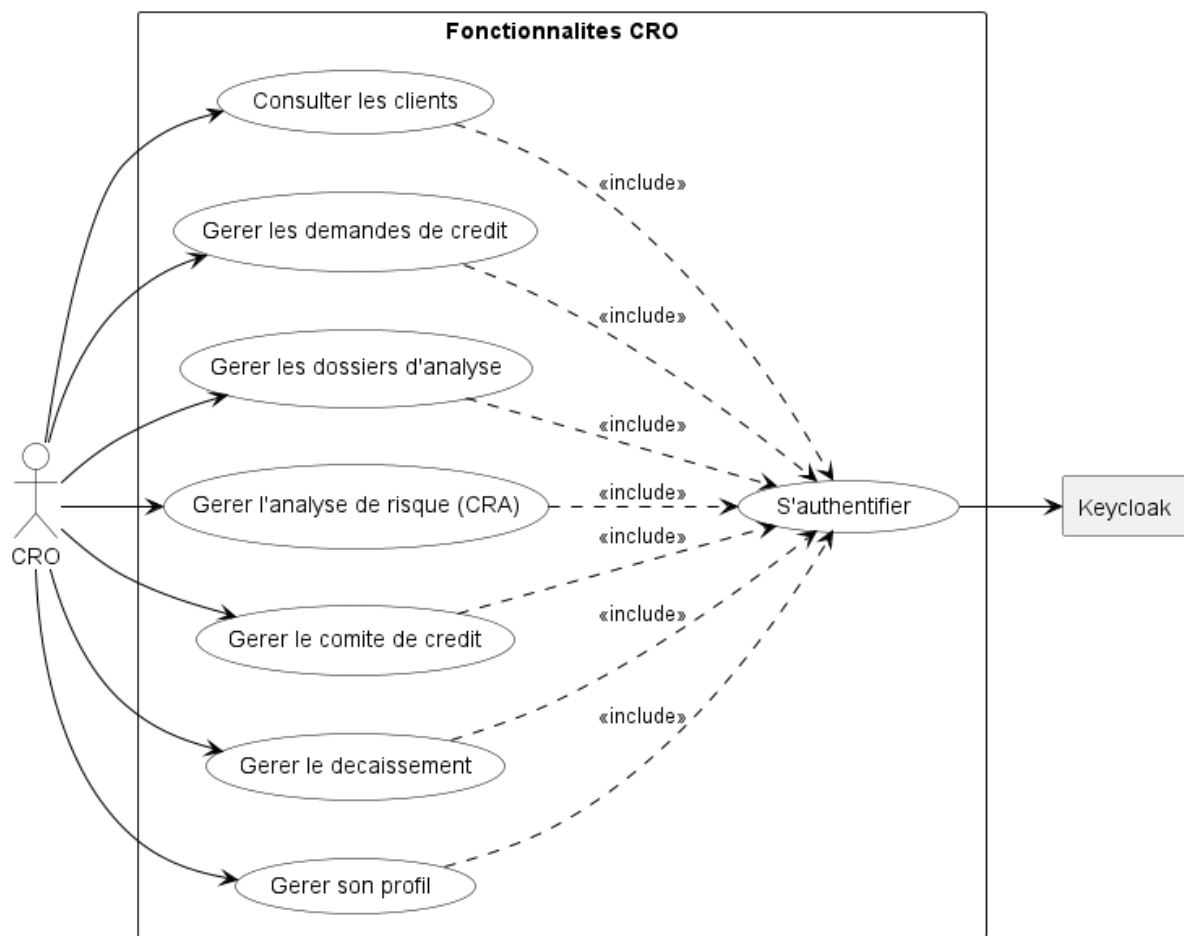


Figure 2.4 : Diagramme de cas d'utilisation — CRO

➤ Diagramme de cas d'utilisation BranchDM

Le diagramme de cas d'utilisation BranchDM (Figure 2.5) présente les fonctionnalités accessibles au décideur agence. Il inclut la gestion des demandes de crédit, la validation des dossiers d'analyse, la gestion du CRA, la prise de décision au comité de crédit et le suivi du décaissement.



Figure 2.5 : Diagramme de cas d'utilisation — BranchDM

➤ Diagramme de cas d'utilisation HeadOfficeDM

Le diagramme de cas d'utilisation HeadOfficeDM (Figure 2.6) présente les fonctionnalités accessibles au décideur de la direction générale. Il inclut la supervision des demandes de crédit, des dossiers d'analyse, du CRA, la validation finale au comité de crédit et le contrôle du décaissement.

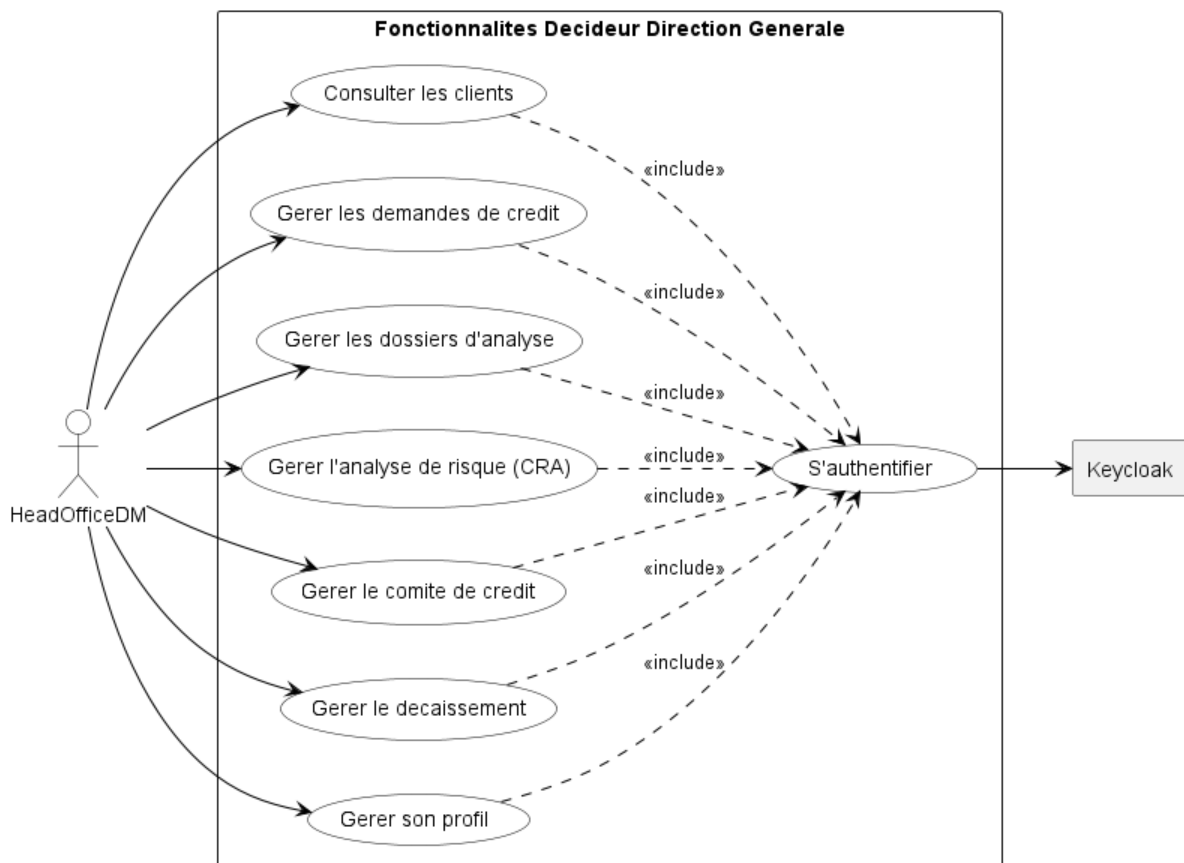


Figure 2.6 : Diagramme de cas d'utilisation — HeadOfficeDM

➤ Diagramme de cas d'utilisation RiskAnalyst

Le diagramme de cas d'utilisation RiskAnalyst (Figure 2.7) présente les fonctionnalités accessibles à l'analyste risque. Il inclut la consultation des demandes de crédit et des dossiers d'analyse, la gestion complète des formulaires CRA, ainsi que la participation au comité de crédit et le suivi du décaissement.

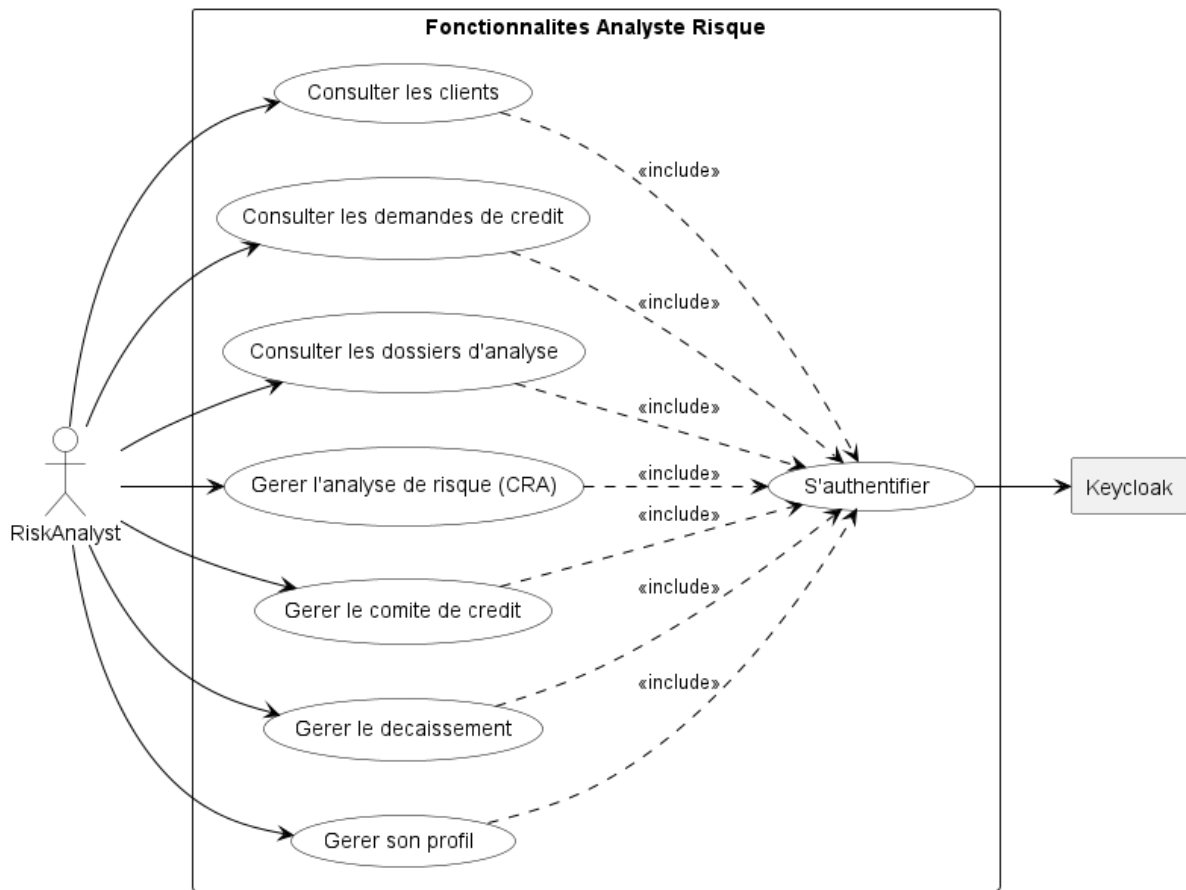


Figure 2.7 : Diagramme de cas d'utilisation — RiskAnalyst

➤ Diagramme de cas d'utilisation ReadOnly

Le diagramme de cas d'utilisation ReadOnly (Figure 2.8) présente les fonctionnalités accessibles aux utilisateurs en lecture seule. Il se limite à la consultation des clients, des demandes de crédit, des dossiers d'analyse, du comité de crédit et du décaissement, sans aucune action d'écriture.

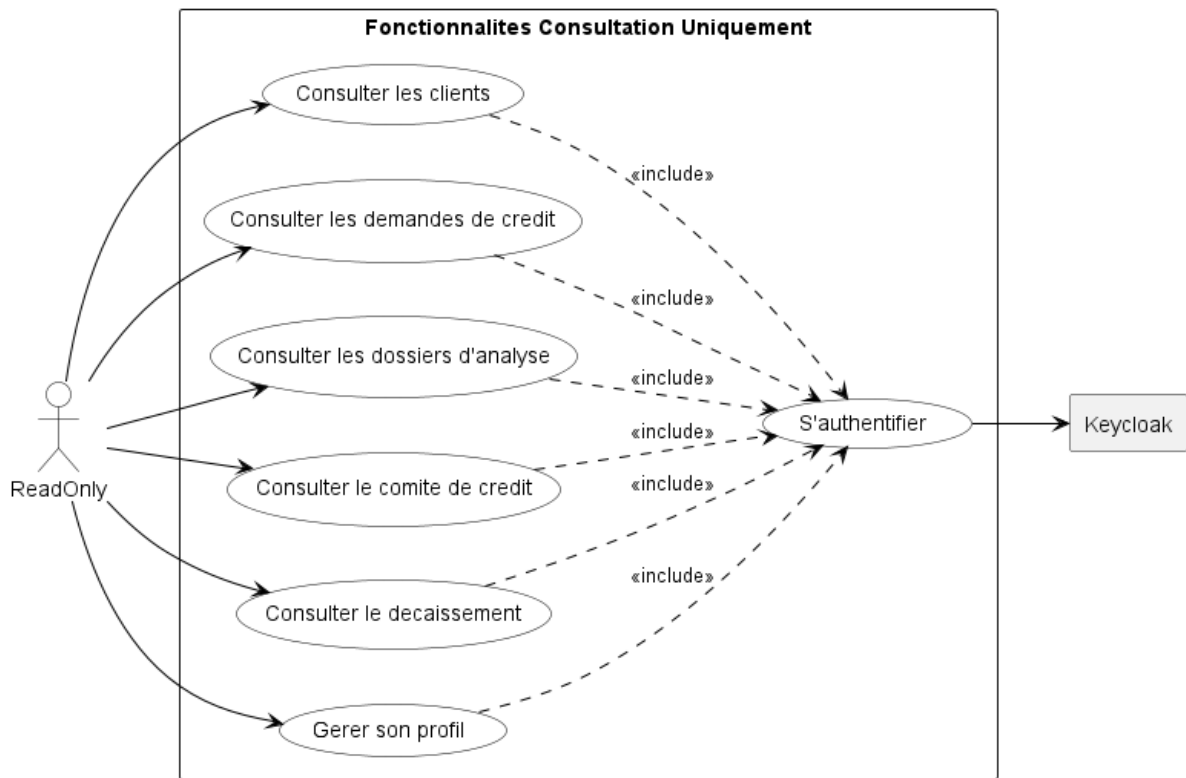


Figure 2.8 : Diagramme de cas d'utilisation — ReadOnly

2.3.2 Diagramme de classes d'analyse

Le diagramme de classes offre une vue détaillée de la conception structurelle de la plateforme CrediWise. Il définit les relations entre les différentes entités du domaine métier, leurs attributs et leurs associations, servant de fondation à l'implémentation du système.

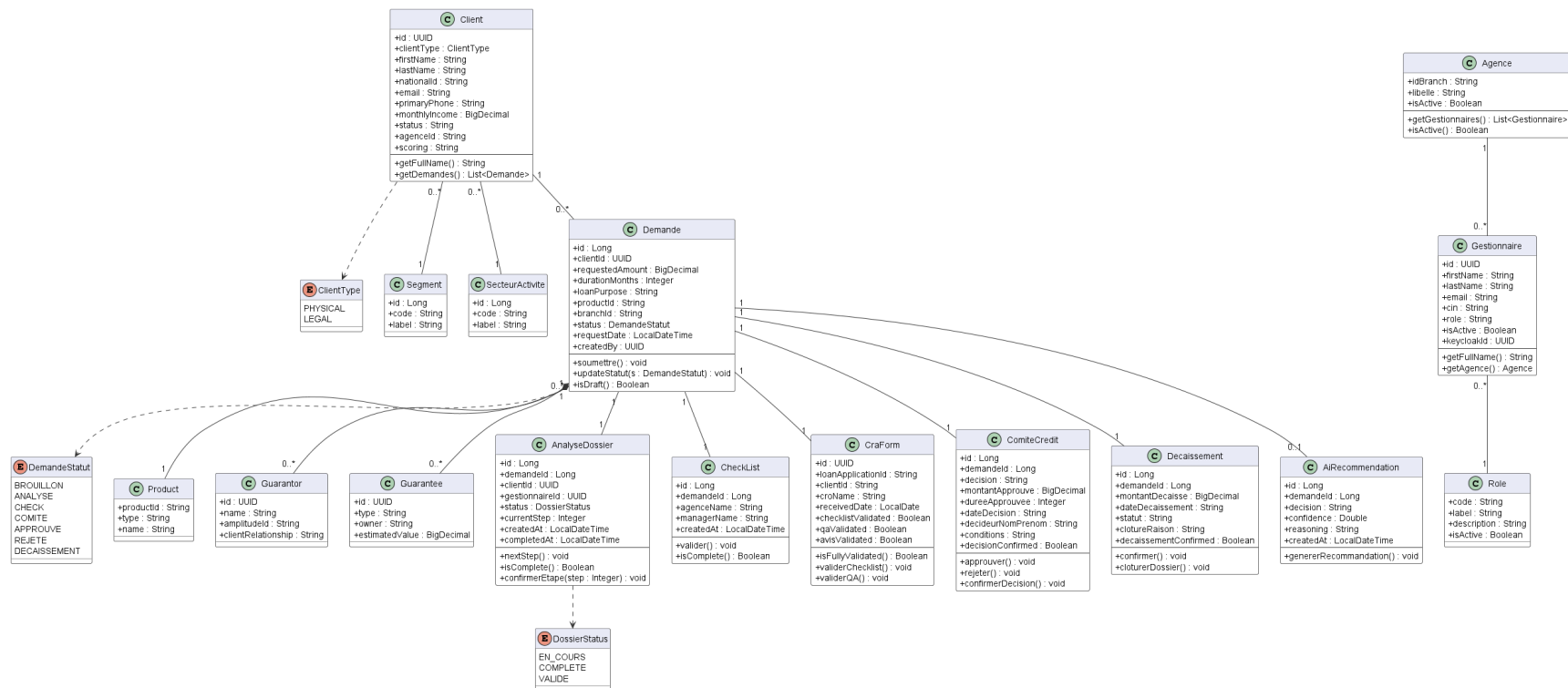


Figure 2.9 : Diagramme de classes global — CrediWise

2.4 Architecture du système

2.4.1 Architecture physique

L'architecture physique met en évidence les différents composants matériels supportant l'application. L'architecture repose sur les composants suivants :

- **Frontend** : L'application React 19 est développée avec Vite et TypeScript. Elle communique avec les services backend via des API REST, intègre Keycloak (keycloak-js) pour l'authentification et utilise des gardes de routes (react-router-dom) pour le contrôle d'accès. Elle embarque également une instrumentation OpenTelemetry pour la traçabilité distribuée de bout en bout.
- **Serveurs Backend (Microservices)** : Dix services indépendants développés avec Quarkus (Java) s'exécutent sur des ports distincts, chacun exposant une API REST documentée via Swagger UI. Quatre d'entre eux exposent également des points d'accès gRPC pour les échanges inter-services :
 - Service Gestionnaire : REST 8080, gRPC 9000 ;
 - Service Client : REST 8082, gRPC 9001 ;
 - Service Nouvelle Demande : REST 8083, gRPC 9002 ;
 - Service Analyse : REST 8084, gRPC 9003 ;
 - Service Checklist : REST 8085 ;
 - Service Risque Analyse : REST 8086, gRPC 9005 ;
 - Service Comité Crédit : REST 8087 ;
 - Service Décaissement : REST 8088 ;
 - Service IA Recommendation : REST 8089, gRPC 9008 ;
 - Service OCR : REST 8000 (Python/FastAPI).
- **Serveurs de Bases de Données** : Chaque microservice maintient sa propre instance PostgreSQL dédiée selon le patron *Database per Service* :
 - `gestionnaire_db` (port 5432)

- `client_db` (port 5433)
 - `nouvelle_demande_db` (port 5434)
 - `analyse_db` (port 5435)
 - `check_list_db` (port 5437)
 - `risque_analyse_db` (port 5438)
 - `comite_credit_db` (port 5439)
 - `decaissement_db` (port 5440)
 - `ai_recommendation_db` (port 5441)
- **Fournisseur d'Identité** : Keycloak 26 est déployé sur le port 8180 avec sa propre base de données PostgreSQL dédiée (port 5436). Il importe le realm CrediWise ainsi que le thème de connexion personnalisé.
- **Observabilité** : SigNoz est déployé comme plateforme de télémétrie (traces, métriques, logs). Un collecteur OpenTelemetry centralise les signaux de tous les microservices et du frontend.
- **Passerelle** : Nginx est configuré comme reverse proxy et passerelle API pour les scénarios entièrement conteneurisés, incluant le TLS, le CORS, les en-têtes de sécurité et les règles de routage vers l'ensemble des dix microservices.

2.4.2 Architecture logique

2.5 Environnement de développement

2.5.1 Environnement logiciel

L'environnement logiciel de CrediWise regroupe l'ensemble des technologies, frameworks et outils utilisés pour le développement, le déploiement et l'exploitation de la plateforme. Le tableau 1.3 présente ces outils ainsi que leur rôle dans le projet.

Outil	Description
Quarkus [2]	Framework Java cloud-native pour les microservices, à démarrage rapide et faible empreinte mémoire.
React [3]	Bibliothèque JavaScript (Meta) utilisée avec Vite et TypeScript pour le frontend SPA de CrediWise.
PostgreSQL [4]	SGBD relationnel open-source robuste ; chaque microservice dispose de sa propre instance dédiée.
Docker / Docker Compose [5]	Plateforme de conteneurisation assurant des environnements de déploiement reproductibles.
Nginx [6]	Reverse proxy et passerelle API gérant le TLS, le CORS et le routage vers les microservices.
FastAPI [7]	Framework Python asynchrone exposant l'API REST du service OCR pour l'extraction de données PDF.
Keycloak [8]	Plateforme IAM open-source centralisant l'authentification, les sessions et la gestion des rôles.
Azure Pipelines [9]	Pipeline CI/CD automatisant la compilation Maven, la construction et la publication des images Docker.
SigNOZ [10]	Plateforme d'observabilité open-source pour les traces, métriques et logs des microservices.
OpenTelemetry [11]	SDK d'instrumentation collectant et exportant les signaux télémétriques vers SigNOZ via OTLP.
Git [12]	Système de contrôle de version distribué pour la gestion collaborative du code source.
IntelliJ IDEA [13]	IDE Java/Quarkus avec assistance au code avancée pour le développement backend.
Visual Studio Code [14]	Éditeur extensible utilisé pour le développement frontend React/TypeScript avec intégration Git.
Postman [15]	Client API pour tester et documenter les API REST des microservices CrediWise.

Outil	Description
pgAdmin [16]	Interface web d'administration des instances PostgreSQL des microservices.
Ollama [17]	Serveur LLM local permettant d'exécuter des modèles de langage open-source (ex. Llama 3.1) comme alternative pour les recommandations IA en environnement hors-ligne.

Tableau 1.3 : Environnement logiciel

2.5.2 Technologies utilisées

- **Quarkus** [2] a été retenu comme framework Java principal pour l'ensemble des services backend en raison de son architecture cloud-native : il réduit significativement l'empreinte mémoire et le temps de démarrage de chaque service, un avantage déterminant lorsque dix microservices s'exécutent simultanément dans des conteneurs. Il unifie dans un seul écosystème cohérent les besoins REST, gRPC, ORM, validation, sécurité OIDC et télémétrie, évitant la dispersion entre plusieurs frameworks indépendants.
- **React** [3] associé à Vite et TypeScript a été choisi côté client pour produire une SPA interactive et maintenable, le typage statique sécurisant les échanges de données avec les API backend.
- **Keycloak** [8] centralise la gestion des identités et libère chaque microservice de toute logique d'authentification propre, assurant une gestion unifiée des rôles et des sessions.
- **PostgreSQL** [4] répond au patron *Database per Service* en offrant un moteur relationnel fiable et éprouvé pour chacune des bases de données isolées des microservices.
- **Docker** et **Docker Compose** [5] garantissent des environnements de déploiement reproductibles du poste de développement jusqu'à la production.
- **Azure Pipelines** [9] automatise l'intégration continue en ne recompilant que les services modifiés à chaque commit, réduisant les temps de build et fiabilisant les livraisons.
- **SigNOZ** [10] et **OpenTelemetry** [11] ont été intégrés pour assurer une observabilité

complète de la plateforme, permettant de corréler traces, métriques et logs à travers l'ensemble des services.

Conclusion

Références bibliographiques

- [1] BOCASAY. « Méthode Scrum : quels sont ses bénéfices pour les développements web ? » [Consulté le 2026]. adresse : <https://www.bocasay.com/fr/methode-scrum-benefices-developpements-web/>
- [2] QUARKUS. « Supersonic Subatomic Java. » [Consulté le 2026]. adresse : <https://quarkus.io/>
- [3] META PLATFORMS, INC. « React Documentation. » [Consulté le 2026]. adresse : <https://react.dev/>
- [4] POSTGRESQL GLOBAL DEVELOPMENT GROUP. « PostgreSQL : The World's Most Advanced Open Source Relational Database. » [Consulté le 2026]. adresse : <https://www.postgresql.org/>
- [5] DOCKER, INC. « Docker Overview. » [Consulté le 2026]. adresse : <https://docs.docker.com/engine/docker-overview/>
- [6] F5, INC. « NGINX Documentation. » [Consulté le 2026]. adresse : <https://nginx.org/en/docs/>
- [7] S. RAMÍREZ. « FastAPI Documentation. » [Consulté le 2026]. adresse : <https://fastapi.tiangolo.com/>
- [8] RED HAT, INC. « Keycloak : Open Source Identity and Access Management. » [Consulté le 2026]. adresse : <https://www.keycloak.org/>
- [9] MICROSOFT. « Azure Pipelines Documentation. » [Consulté le 2026]. adresse : <https://learn.microsoft.com/en-us/azure/devops/pipelines/>
- [10] SIGNOZ, INC. « SigNOZ : Open Source Observability Platform. » [Consulté le 2026]. adresse : <https://signoz.io/>
- [11] CLOUD NATIVE COMPUTING FOUNDATION. « OpenTelemetry Documentation. » [Consulté le 2026]. adresse : <https://opentelemetry.io/docs/>

- [12] SOFTWARE FREEDOM CONSERVANCY. « Git Documentation. » [Consulté le 2026].
adresse : <https://git-scm.com/doc>
- [13] JETBRAINS. « IntelliJ IDEA. » [Consulté le 2026]. adresse : <https://www.jetbrains.com/idea/>
- [14] MICROSOFT. « Visual Studio Code Documentation. » [Consulté le 2026]. adresse :
<https://code.visualstudio.com/docs>
- [15] POSTMAN, INC. « Postman API Platform. » [Consulté le 2026]. adresse : <https://www.postman.com/product/>
- [16] PGADMIN DEVELOPMENT TEAM. « pgAdmin : PostgreSQL Tools. » [Consulté le 2026].
adresse : <https://www.pgadmin.org/>
- [17] OLLAMA. « Ollama : Run Large Language Models Locally. » [Consulté le 2026]. adresse :
<https://ollama.com/>